MiniProject 3

Seyed Yazdan Zinati SeyedBehnam Mashari Mohammadhadi Foroutan

April 1, 2021

Abstract

1 Introduction

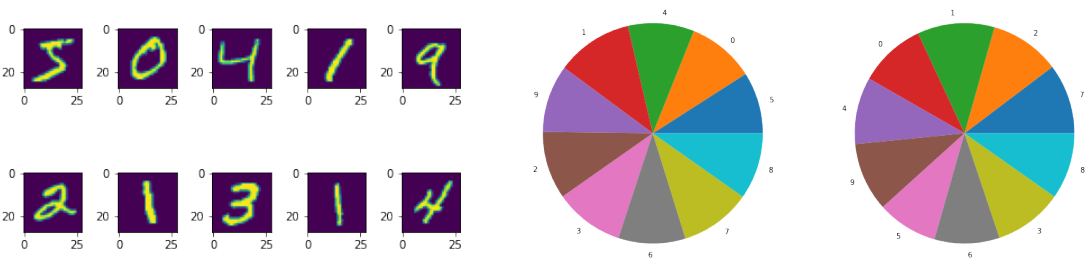
Contrary to the machine-generated characters and digits, which are uniform throughout documents, each person has a specific writing style. handwritten text recognition is a subarea in intelligent image analysis and is a practically engrossing problem especially, owing to its wide range of applications from check processing [1] to postal mail sorting. Automatic recognition of handwritten digits, characters and symbols through machine learning algorithms are investigated in [2]. Different classification algorithms, namely KNN, Naive Bayes, and K Star algorithms were deployed in [3]. However, many research papers have focused on using deep learning approaches such as Artificial Neural Networks (ANN), specifically Multilayer Perceptrons (MLPs). These networks are inspired by the structure of the human brain and consist of different layers, namely input, output and hidden layers.

In this project, we first acquire vectorize and perform mean subtraction and normalization of MNIST handwritten digit data set. Next, we implement a MultiLayer Perceptron (MLP) from scratch to classify the above-mentioned data set. Our implementation of the MLP allows a flexible number of hidden layers and neurons in each layer. It can be configured to run with one of four implemented activation functions (Sigmoid, Hyperbolic Tangent, ReLU and LeakyReLU), L2 regularization (weight decay), and uses mini-batch stochastic gradient descent SGD. Different software components were also added to tune, configure, test, and visualize models and the corresponding results.

Using the described MLP, we run multiple experiments. First, we study the effect of network depth on accuracy, convergence, underfitting and overfitting. We run this experiment for three models, with varying depths from zero to two. We then expand this experiment and vary network depth and width to find the most accurate and performant model. This study is followed by a study of non-linearity and a comparative analysis of the four mentioned activation functions and the effect on the MLP. With the aim of finding the best-performing network configuration, we study the outcome of training the network using unnormalized images and adding L2 regularization to the cost. We finally gather our findings to implement the best-performing model into a graphical user interface to classify digits drawn with mouse inputs.

2 Datasets

Containing 60,000 training and 10,000 testing images of handwritten digits, the MNIST handwritten digit dataset is a size-normalized large dataset, having fit and centred into 28×28 pixels images. There are ten labels corresponding to these handwritten digits from 0 to 9. We loaded the MNIST dataset from the `tf.keras.datasets` module and used its default train and test partitions. As a common way of understanding datasets, we visualized samples of the MNIST dataset digits as can be seen in Figure 1 (a). Moreover, train and test data distributions, illustrated in Figure 1 (b) (c), provide decent evidence that the dataset is unbiased.



(a) MNIST dataset samples (b) Train data class distribution (c) Test data class distribution

Figure 1

We started preprocessing by vectorizing and flattening the data to obtain appropriate dimensions so that training and testing samples are converted into $60,000 \times 784$ and $10,000 \times 784$ arrays, respectively. Then, we used mean subtraction as a routine part of the preprocessing pipeline, where the mean is subtracted from every pixel in the resulting array. It is the same interpretation of zero-centring of data clouds. Although it is not an utter necessity to perform normalization for image datasets, unnormalized data contains a large difference in magnitudes between inputs, slowing the learning process. We study the effect of unnormalized inputs in more depths in the following section. Therefore, as the next step, we divided data samples associated with every instance (each digit or each row in the array) to the standard deviation of that instance. Since our data has passed through the mean subtraction phase, the resulting values are between -1 and +1.

There are several other steps in the conventional preprocessing of image datasets, including removing black pixels, dimensionality reduction, and whitening, all of which have been implemented but not used in our preprocessing. Black pixels are removed to get rid of non-informative pixels to speed up computations, PCA technique reduces dimensionality using the top few eigenvectors of the data covariance matrix, and whitening operation divides every data dimension by its eigenvalue to normalize the scale. It is noteworthy that these techniques can be deployed for further studies around MNIST dataset analysis in the future.

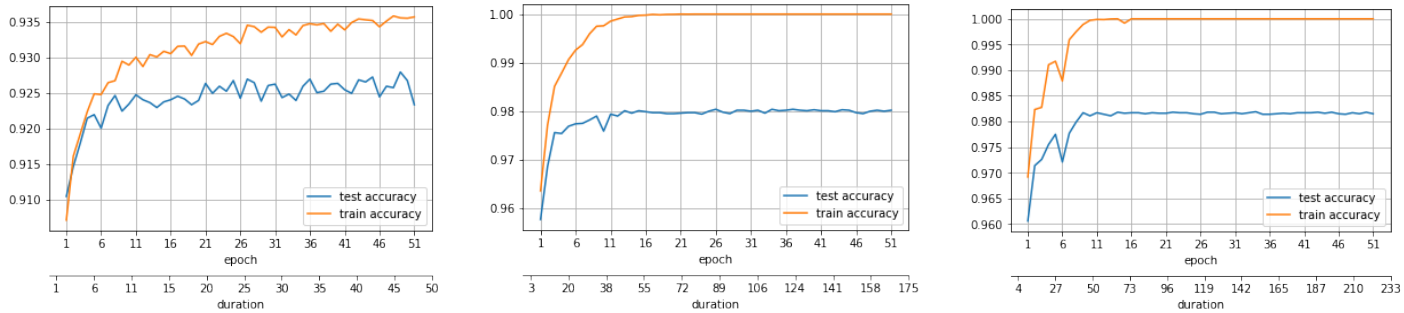
3 Results and Analysis

We implement a Multi-Layer Perceptron (MLP) from scratch. The number of input, output, and hidden layer units, as well as the number of hidden layers, can be configured. As the input to the MLP is a flattened 28×28 two-dimensional array, we configure the MLP to take 784 (28×28) input units. We are classifying digits 0-9, thus we configure the MLP to have 10 output units. For the MLP to learn non-linear decision boundaries, we introduce non-linearity to our network by implementing four activation functions (Sigmoid, TanH, ReLU, and Leaky ReLU). We use mini-batch stochastic gradient descent as the optimization algorithm in our backpropagation. Additionally, to tackle the problem of exploding or vanishing gradients in our network, we assure equal variance between layers using Xavier initialization to initialize weights in our deep layers. We follow tutorials [4, 5, 6] in implementing parts of the Multilayer Perceptron.

3.1 Network Depth and Number of Neurons in the Hidden Layer

We ran experiments to study the effect of learning rate and batch size parameters on the convergence rate of the network. These as well as other additional experiments can be found in the submitted notebook. We concluded that small batch sizes and large learning rates converge quickly with the cost of oscillation and noise during the learning process, whereas large batch sizes and small learning rates converge slowly with more precise estimates of the loss gradient. Therefore, we tune hyperparameters for each model to balance the learning rate and batch size.

We run three experiments with three different models and plot the train and test accuracies as a function of epochs. For the first model, we create an MLP with no hidden layers that directly maps the inputs to outputs. The second model contains a single hidden layer having 128 units and ReLU activations. The third model MLP is similar to the second one with one additional 128 unit hidden layer. Before running each experiment, we tune model hyperparameters including batch size and learning rate by performing a grid search. The details of this process and results can be found in the submitted notebook. The results of our experiments can be found in Figure 2.



(a) No hidden layers

(b) One 128-unit hidden layer with ReLU activation

(c) Two 128-unit hidden layers with ReLU activation

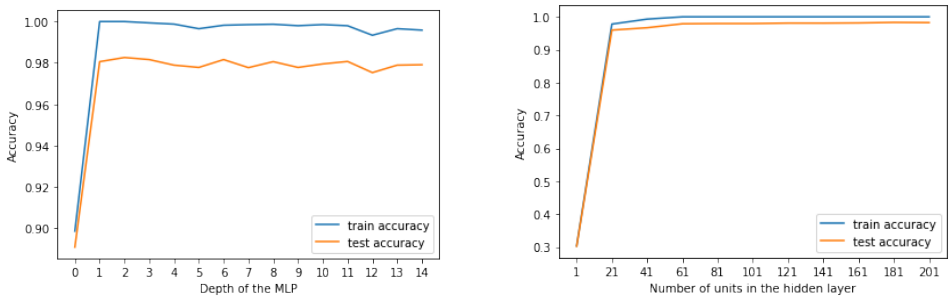
Figure 2: Test and train performance as a function of training epochs

For the model with no hidden layers, a learning rate of 0.01 and batch size of 100 results in the best accuracies. Using those hyperparameters, we observe in Figure 2 (a) that although train and test accuracies oscillate considerably between epochs, the accuracies have a positive trend overall. However, these oscillations rise considerably using a larger learning rate and/or a smaller batch size. However, even after 50 epochs, the model fails to fully converge and reach relatively constant train and test accuracies. The model reaches train and test accuracies of approximately 93% and 92% after 51 epochs respectively.

Tuning the learning rate and batch size of the model with one hidden layer results in a learning rate of 0.1 and batch size of 70. As can be seen in Figure 2 (b), the model converges noticeably faster with little oscillation compared to the former model. The model approximately reaches the maximum train and test accuracy after 16 epochs and train and test accuracies of roughly 100% and 98% after 51 epochs respectively.

We use a tuned learning rate and batch size of 0.1 and 50 for the third model with two hidden layers. Similar to the model with one hidden layer, this model shown in Figure 2 (c), converges with little oscillation and reaches the same train and test accuracies but converges faster after only 10 epochs. Even though the train and test accuracies are similar in the model with one and two hidden layers, the time to train the network is higher in the latter (223 vs 175 seconds) as the training time is proportional to the number of hidden layers in our network. Moreover, we note that although the training accuracy reaches 100% in the models with one and two hidden layers, the test accuracy never declines. Therefore, we don't see any considerable signs of overfitting within the number of epochs we experimented.

To further study the effect of network depth on the accuracy, we plot the train and test accuracies of networks with depths 0 to 14 shown in Figure 3 (a). We note an enormous boost of train and test accuracy from depth 0 to 1. However, increasing network depth from 1 to 14 doesn't remarkably affect accuracy. In our case increasing the number of hidden layers does not result in overfitting. The acquired results are in line with [7, 8] as a neural network with a single hidden layer with a finite number of units in the hidden layer can approximate continuous functions according to the universal approximation theorem. We obtained a lower accuracy using the MLP with no hidden layers as it can simply approximate linearly separable functions, whereas the MLP with one hidden layer can represent any decision boundary with the ReLU activation function. The network containing a single hidden layer offers great accuracy with low time complexity. Moreover, as we mentioned, it is capable of approximating arbitrary continuous functions according to [8] with a finite number of neurons. Therefore, we run an experiment to determine the ideal number of neurons in its hidden layer. We increase the number of neurons in the hidden layer from 1 to 201 in increments of 20 and plot the corresponding test and train accuracies. The results of this experiment illustrated in Figure 3 (b) shows substantial growth in accuracy when increasing the number of neurons in the hidden layer from 1 to 20 (30% to 95%). Test and train accuracies increase steadily until they plateau after 61 neurons in the hidden layer when the model is no longer underfitting. Increasing the number of neurons could result in overfitting, which is not the case in this study.



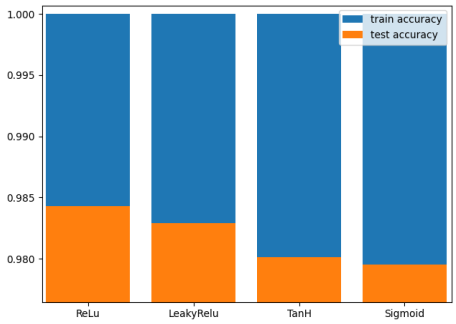
(a) Test and train performance as a function of network depth

(b) Test and train performance as a function of number of neurons in the hidden layer

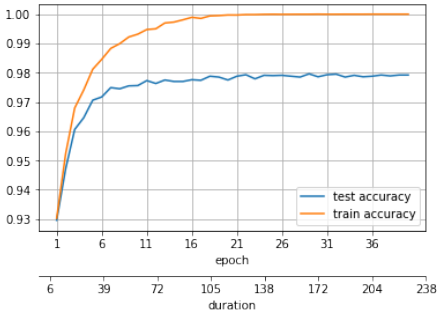
Figure 3

3.2 Non-linearity and Activation Functions

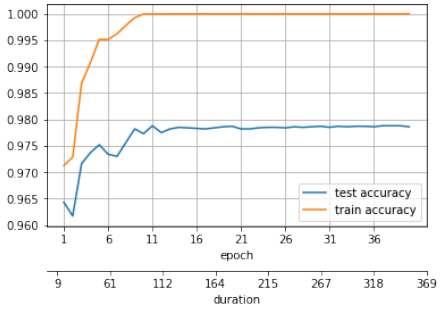
In this part, we evaluate the performance of an MLP containing two hidden layers, each having 128 neurons using different four activation functions (Sigmoid, TanH, ReLU, LeakyReLU) after training them for 40 epochs. The results of this experiment are shown in Figure 4. As mentioned previously, we need activation functions to introduce non-linearity to the neural network and to be able to create a universal approximator. We achieve the highest final test accuracy using ReLU, followed closely by LeakyReLU with 98.4% and 98.3% accuracy. TanH comes at third place with a test accuracy of 98.1%, closely followed by the Sigmoid activation with 98.0% test accuracy. Our results conform to the theory as the sigmoid function, which performs the worst in our experiment is prone to vanishing gradient during which the learning is minimal. It can also be concluded from Figure 4 (b) that the model using Sigmoid activation converges slower compared to TanH or ReLU activations. Similar to the Sigmoid activation the Hyperbolic Tangent activation is also prone to vanishing gradient. However, the derivative is steeper compared to the sigmoid function due to being defined between -1 and 1 (instead of 0 and 1). The TanH activation function also has the worst time complexity among activation functions. It should be noted that initializing weights using Xavier initialization has to a moderate extent reduced the problem with vanishing gradients. The ReLU activation function performs the best with the smallest time complexity and is not prone to vanishing gradients. ReLU slightly outperformed LeakyReLU, due to not facing the dying ReLU problem during the learning process and randomness.



(a) Comparison of the MLP accuracy using different activation functions



(b) Performance of the MLP using Sigmoid activation



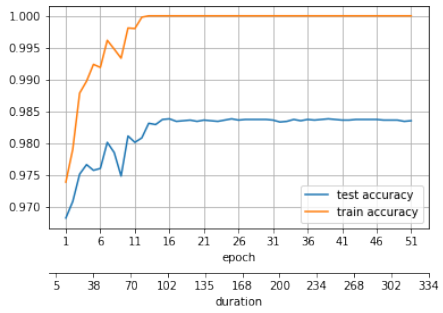
(c) Performance of the MLP using TanH activation

Figure 4

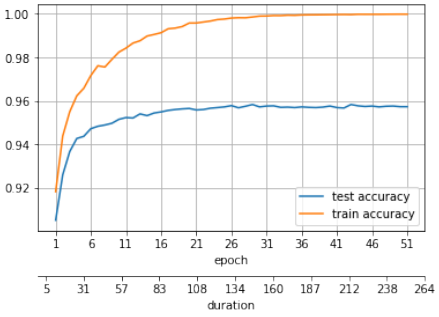
3.3 L2 Regularization (Weight Decay)

We study the consequences of adding L2 regularization (weight decay) to the cost functions of the MLP by plotting the accuracy as a function of epochs of an MLP with ReLU activation and two hidden layers, each layer containing 128 neurons. We achieve the best results when λ is equal to 0.1. The results of this experiment can be found in Figure 5 (a). Due to the large solution space of deep neural networks, the network could overfit, hence failing to generalize. L2 regularization reduces generalization error by preventing weights to become large, thus promoting simpler models.

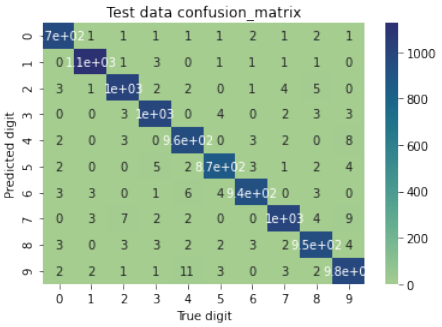
As pointed out previously, the test accuracy of the MLP does not decay with larger training epochs we have experimented with (51 epochs). Consequently, the obtained results with L2 regularization is similar to the model without regularization applied. However, if the model overfitted, meaning that the test accuracy decreased after reaching a maximum, weight decay would have prevented this decrease from happening.



(a) Performance of the MLP with L2 regularization (weight decay)



(b) Performance of the MLP using unnormalized images



(c) Confusion matrix of the MLP using unseen data

Figure 5

3.4 Normalization

As mentioned previously, as part of the preprocessing of the dataset, we normalize images. To study the effect of training the MLP using unnormalized images on accuracy, we use the previously described two-layer MLP with ReLU activation.

Running the experiment using unnormalized images pointed out the numerical instability in our implementation of the loss and activation functions. Due to these functions involving many exponential functions, large unnormalized inputs caused errors such as overflows. After applying the appropriate fixes to ensure numerical stability with unnormalized images, we ran the experiment. Similar to the previous experiment, we tune the minibatch size and learning rate hyperparameters before running the experiment. Contrary to the MLP trained using normalized images, the MLP trained using unnormalized images failed to converge under a reasonable number of epochs with a learning rate of 0.1. Interestingly, it performed best when using a smaller learning rate of 0.001. The plot of train and test performances of the MLP trained with unnormalized images is illustrated in Figure 5 (b). We can see that the MLP trained using unnormalized images not only takes a longer time to converge but also settles at a smaller train and test accuracy of 99% and 95% compared to the MLP trained using normalized images with train and test accuracies of 100% and 98%.

An essential reason we achieve a superior result when normalizing the input to the MLP is that the large difference in magnitude of inputs causes the weights associated with some inputs to be updated faster than other ones, thus negatively affecting the learning in our network. Normalization thus deals with this problem by scaling the input. Moreover, as we

apply mean subtraction to image data in the preprocessing, input data is between -1 and 1, thus avoiding zigzag weight updates (weights all increasing or decreasing) when for example using Sigmoid activation.

3.5

To make use of all of our findings, we create a graphical user interface and integrate into it our most performant model. This GUI accepts mouse input to draw a digit on the canvas and identifies the drawn digit. The GUI can be launched by running `gui.py`. Demonstrations of this program can be found in the submitted `demo` folder.

By creating the confusion matrix (Figure 5 (c)) of the most performant network using unseen data, we can extract information such as the similarities between handwritten digits. We find that digit pairs (4, 9) and (7, 9) are the most confused pairs by our network and can have high similarities when drawn.

4 Discussion and Conclusion

To conclude, we implemented a MultiLayer perceptron to classify image data from the MNIST handwritten digit dataset. We preprocessed the dataset and ran multiple experiments on network depth, the number of neurons in hidden layers and interpreted our results using the universal approximation theorem and [7, 8]. We found that a network with one hidden layer offered great advantages compared to a network with no hidden layer, a network with two or more hidden layers did not offer considerable advantages to the former. Additionally, we proved the possibility of achieving high accuracy by using a finite number of neurons in a one-layer network. Next, we ran analyses on different activation functions and found that ReLU offered the best learning time, faster convergence, and the highest accuracy. We found that even though L2 regularization did not significantly affect the performance of our model, training the network with normalized images, resulted in a notable boost in accuracy and covergence time. Finally, we compiled all of our findings on the network and dataset in a GUI capable of classifying digits drawn by mouse input on a canvas. Our future works will build on the current implementation and study the results of and previously mentioned additional preprocessing methods, and dropout and data augmentation on the network.

Statement of Contribution

This study consisted of implementing different software component including the MLP, loss functions, dataset and result visualizers, the GUI, and model analyzers done by Seyed Yazdan Zinati. Seyedbehman Mashari implemented the preprocessor and regularization. Mohammadhadi Foroutan wrote to the Introduction section in the report. Seyedbehnam Mashari wrote the Dataseta section, and Seyed Yazdan Zinati compiled results and findings in the Results and Analysis and Conclusion section.

References

[1] R. Jayadevan, S. R. Kolhe, P. M. Patil, and U. Pal, "Automatic processing of handwritten bank cheque images: a survey," *International Journal on Document Analysis and Recognition (IJDAR)*, vol. 15, no. 4, pp. 267–296, 2011.

[2] S. M. Shamim, M. B. Miah, A. Sarker, M. Rana, and A. A. Jobair, "Handwritten Digit Recognition Using Machine Learning Algorithms," *Indonesian Journal of Science and Technology*, vol. 3, no. 1, p. 29, 2018.

[3] S. HAMIDA, B. CHERRADI, A. RAIHANI, and H. OUAJJI, "Performance Evaluation of Machine Learning Algorithms in Handwritten Digits Recognition," *2019 1st International Conference on Smart Systems and Data Science (ICSSD)*, 2019.

[4] A. Agrawal, "Building Neural Network from scratch" *Medium*, 18-Feb-2019. [Online]. Available: https://towardsdatascience.com/building-neural-network-from-scratch-9c88535bf8e9. [Accessed: 31-Mar-2021].

[5] Lzkatz, "How do I implement leaky relu using Numpy functions" *Stack Overflow*, 24-May-2018. [Online]. Available: https://stackoverflow.com/a/50517613. [Accessed: 31-Mar-2021].

[6] V. Valkov, "Creating a Neural Network from Scratch-TensorFlow for Hackers (Part IV)," *Medium*, 20-Jun-2018. [Online]. Available: https://medium.com/@curiousily/tensorflow-for-hackers-part-iv-neural-network-from-scratch-1a4f504dfa8. [Accessed: 31-Mar-2021].

[7] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals, and Systems*, vol. 2, no. 4, pp. 303–314, 1989.

[8] K. Hornik, "Approximation capabilities of multilayer feedforward networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991.